

MICROSAR Classic Fee_30_FlexNor

Technical Reference

Version 4.04.01

Contents

1	Introduction	8
1.1	Architecture Overview	8
1.2	Flash Layout Compatibility	9
2	Functional Description	10
2.1	Features	10
2.1.1	Backward Compatibility for Flash Driver	11
2.2	Limitations	11
2.2.1	Maximum Logical Sector Size	11
2.2.2	Memory Access Module	11
2.3	Error Handling	11
2.3.1	Development Error Reporting	11
2.3.2	Production Code Error Reporting	11
2.4	Compliance	11
2.4.1	Erase of Immediate Blocks	12
2.4.2	Reference to MEMACC Address Area	12
2.5	Flash Organization	12
2.5.1	Partitions	13
2.5.2	Logical Sectors	14
2.5.2.1	Effects of Logical Sector Count	14
2.5.3	Layouts	14
2.5.3.1	Secure Layout	15
2.5.3.2	Slim Layout	15
2.6	Lookup Table	16
2.6.1	Persist Lookup Table	16
2.6.2	Load Lookup Table	16
2.6.3	Recover Lookup Table	17
2.7	Garbage Collection	18
2.8	Block Configuration Update	18
2.9	Partition Configuration Update	19
2.10	Cancellation	19
2.11	Bootloader Mode	19
2.12	Suspend and Resume Internal Recovery Jobs	20
2.13	Suspend and Resume Write Related Jobs	21
2.14	GetEraseCycleCounter	21
2.15	Exception Handling	22
2.15.1	Milestones and Exceptions	22

2.15.1.1	Milestones	23
2.15.1.2	Errors and Warnings	23
2.15.2	Integration	24
2.15.2.1	Callback Implementation	24
2.15.2.2	Configuration	25
2.15.3	Example Implementation of the User Callback	25
3	Integration	26
3.1	Embedded Implementation	26
3.2	Initialization	29
3.3	Main Functions	29
3.4	Callouts	30
3.4.1	Lower Layer Interaction	30
3.4.2	Upper Layer Interaction	30
3.4.2.1	NvM	30
3.4.2.2	User	31
3.5	Synchronization	31
3.5.1	Critical Sections	31
3.6	Memory Sections	31
3.7	MemMap	31
3.8	Module Det	31
3.9	Module Fls	31
3.10	Module MEMACC	32
3.11	Dependencies on HW Modules	32
3.12	Error Reporting Det	32
3.13	Requirements and Recommendations	32
3.13.1	Requirements for the User System	32
3.13.2	Requirements for the Flash Memory and the Driver	33
3.13.2.1	General Requirements	33
3.13.2.2	Write-Related	33
3.13.2.3	Read-Related	33
3.13.2.4	Erase-Related	33
3.13.2.5	Recommendations	34
3.13.3	Update Information	34
4	API Description	35
4.1	Provided Functions	35
4.1.1	Fee_30_FlexNor_Init	35
4.1.2	Fee_30_FlexNor_SetMode	35

4.1.3	Fee_30_FlexNor_Read	36
4.1.4	Fee_30_FlexNor_Write	37
4.1.5	Fee_30_FlexNor_InvalidateBlock	38
4.1.6	Fee_30_FlexNor_EraseImmediateBlock	38
4.1.7	Fee_30_FlexNor_PersistLookupTable	39
4.1.8	Fee_30_FlexNor_Cancel	40
4.1.9	Fee_30_FlexNor_GetStatus	40
4.1.10	Fee_30_FlexNor_GetJobResult	41
4.1.11	Fee_30_FlexNor_SuspendRecoveryJobs	41
4.1.12	Fee_30_FlexNor_ResumeRecoveryJobs	42
4.1.13	Fee_30_FlexNor_SuspendFlashWrites	43
4.1.14	Fee_30_FlexNor_ResumeFlashWrites	43
4.1.15	Fee_30_FlexNor_GetEraseCycleCounter	44
4.1.16	Fee_30_FlexNor_MainFunction	44
4.1.17	Fee_30_FlexNor_GetVersionInfo	45
4.1.18	Fee_30_FlexNor_JobEndNotification	45
4.1.19	Fee_30_FlexNor_JobErrorNotification	45
4.1.20	Fee_30_FlexNor_AddressAreaJobEndNotification	46
4.2	Required Functions	47
5	Configuration	48
5.1	Configuration Variants	48
5.2	Configuration with DaVinci Configurator	48
5.2.1	Block Configuration	48
5.2.1.1	BlockNumber and BlockId	48
5.2.1.2	Dataset Blocks	48
5.2.1.3	Lookup Table Blocks	49
5.2.2	Partition Configuration	50
5.2.2.1	Reference to Flash Access Layer	50
5.2.2.2	Alignment Information	51
5.2.2.3	Logical Sectors	52
5.2.3	FEE General Configuration	52
5.2.3.1	Internal Buffer Size	52
5.2.4	Flash API Configuration	52
5.2.4.1	Fls_BlankCheck	53
6	Glossary and Abbreviations	54
6.1	Glossary	54
6.2	Abbreviations	54

Document Information

History

Version	Date	Author	Remarks
1.00.00	2021-10-25	virlljs	Initial creation
1.01.00	2022-01-07	virlla, virjwa	Load and persist lookup table
1.02.00	2022-04-05	virjwa, virlla, vircre	Added internal buffer size explanations, information about MemMap generation, NvM callback notification information, block configuration update information and bootloader mode information
2.00.00	2022-12-08	virbka	Updated mainly configuration chapter
2.01.00	2023-04-06	virhdr	Updated integration chapter to highlight restrictions regarding the execution of MainFunction and API services in the same process context
2.01.01	2023-05-30	twagenpfeil	Added information to the MemMap section about making Memory Mapping compatible to older releases.
3.02.00	28.11.2023	twagenpfeil	Remove limitation about the MemAcc as lower layer
3.05.00	26.06.2024	twagenpfeil	Update Persist lookup table section
3.06.00	22.08.2024	speter	Added description of API to suspend and resume write related jobs of the lower layer.
3.06.00	29.08.2024	speter	Added description of API to suspend and resume internal recovery jobs.
3.07.00	02.10.2024	twagenpfeil	Added description of API to get erase cycle counter.
4.00.00	02.12.2024	twagenpfeil	Update Persist lookup table section

Table 1 History

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	Specification of Flash EEPROM Emulation	R22-11
[2]	AUTOSAR	Specification of Development Error Tracer	V4.3.0
[3]	AUTOSAR	List of Basic Software Modules	V4.3.0
[4]	MICROSAR	MemMap Technical Reference	V4.3.0
[5]	AUTOSAR	Specification of Flash Driver	R23-11
[6]	AUTOSAR	Specification of Memory Access	R23-11
[7]	AUTOSAR	AUTOSAR_CP_SWS_FlashEEPROMEmulation.pdf	R23-11

**Caution**

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

1 Introduction

This document describes the functionality, integration, API and configuration of the module Fee.

The Fee is specified in AUTOSAR [1].

The FEE provides access to a dedicated flash area for storing data persistently. It is intended to be used exclusively either by the NVRAM Manager or by a SW instance within a Flash-Boot-Loader.

Further on, the module depends on some other modules, like DET for error handling, the underlying Flash driver for hardware access and the MEMIF for consistent types.

1.1 Architecture Overview

These are the interfaces of the module:

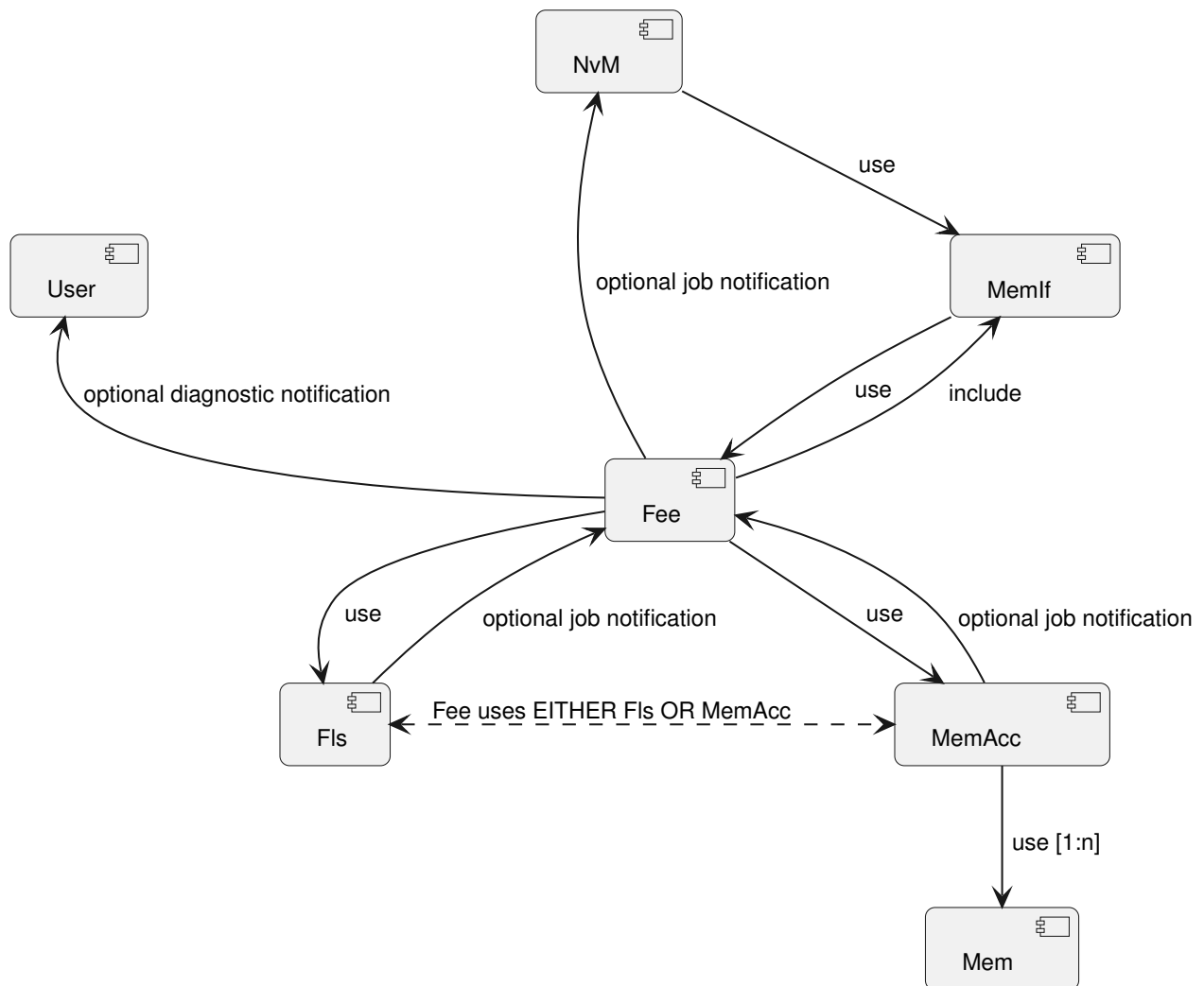


Figure 1.1 Module Architectural Environment

The FEE can either forward memory request to an AUTOSAR FLS [5] or an AUTOSAR MEMACC [6] address area.

Note that MEMACC can also have other users (e.g. Software Download Stack) which access other MEMACC address area's

**Caution**

FEE assumes exclusive usage of FLS or of a MEMACC Address Area. Allowing other components to use FLS's services or the same MEMACC Address Area results in synchronization issues, which are hard to solve. Hence this shall be avoided.

1.2 Flash Layout Compatibility

When updating to a new version of FEE or if data shall be shared between two FEEs (e.g. bootloader and application), it shall be checked by the user, that the two involved FEEs are compatible.

2 Functional Description



Info

Feature specification references (*"This feature was initially created considering..."*) shall provide transparency relative to the specification we considered while implementing.

This information does not allow for conclusions relative to compliance. For compliance / deviation information, instead refer to the subsequent compliance chapter.

2.1 Features

The following features specified in [1] are supported:

- > Reading, writing, and invalidating configured blocks in flash.
- > Abstraction from flash specific restrictions like erase cycles, physical sector size, and alignments.
- > Increased robustness due to detection of incomplete write operations caused by e.g., resets or cancellation.
- > Increased robustness due to detection of unsuccessful write operations caused by e.g., flash job failures or rejects.
- > Cancellation of ongoing jobs (see Section 2.10).
- > Callback notification for completed flash jobs.

The following sub-chapters show additional features provided beyond the AUTOSAR standard:

- > Provision of different flash layouts to adapt to different types of flash devices (see Section 2.5.3).
- > Lookup table to speed up internal search operations (see Section 2.6).
- > Automated garbage collection that removes outdated payloads (see Section 2.7).
- > Automated chunk sizing depending on the write frequency of blocks.
- > Support of operation in bootloader mode where only an incomplete block configuration is provided.
- > Backward compatibility for Flash Driver. See Section 2.1.1 for details.
- > Partitions, that enable the separation of blocks into different and independent flash regions.
- > Configuration of 2 to 32 logical sectors per partition to optimize the data management behavior.
- > Possibility to suspend and resume internal recovery jobs to e.g. improve performance during startup.

- > Possibility to suspend and resume write/erase jobs of the lower layer to e.g. recognize low voltage periods.
- > Possibility to get the maximal erase cycle counter of all sectors for a provided partition.
- > Callback notification for exceptions and debug information.

2.1.1 Backward Compatibility for Flash Driver

Within AUTOSAR 22-11 [1] the usage of the FLS is not specified anymore. Despite that FEE supports besides MEMACC still also the FLS and therefore maintains backward compatibility to older AUTOSAR versions and FLS in general.

2.2 Limitations

2.2.1 Maximum Logical Sector Size

The configurable size of a logical sector is restricted by the size of the internally used chunk links, which are 4-bytes wide, but one byte is used to encode the sector id. Therefore only 3-bytes are available for addressing chunks. Due to an optimization, by addressing pages instead of bytes, the maximum configurable logical sector size in bytes is given by:

$$\text{max sector size} = 2^{24} \cdot (\text{configured page alignment})$$

2.2.2 Memory Access Module

FEE can forward requests either to a FLS or to a MEMACC module.

2.3 Error Handling

2.3.1 Development Error Reporting

The module supports development error detection and reporting using the AUTOSAR Det. If enabled, these errors are reported using the service Det_ReportError().

The reported service IDs and error IDs are contained and documented in the module header file.

2.3.2 Production Code Error Reporting

FEE does not report any production related error to an error/event manager like DEM.

2.4 Compliance

These are the major unsupported features compared to AUTOSAR [1]

- > Erasing immediate blocks to speed up the next write. See Section 2.4.1 for details.
- > Read requests will not be aligned according to the configured `FeeMinimumReadPageSize`.
- > Abstracting from physical flash properties using the virtual page size.

- > Blocks cannot be mapped to different MEMACC AddressAreas via `FeeMemAccAddressArea`. See Section 2.4.2 for details.

2.4.1 Erase of Immediate Blocks

Although the FEE provides the API `Fee_30_FlexNor_EraseImmediate()`, the blocks are not erased in flash. Therefore, the next write request for the block is not completed faster when calling the erase immediate service before.



Note

The block is not really erased in flash but instead it is marked as invalid. After calling `Fee_30_FlexNor_EraseImmediate()` for a block, a consecutive read job will deliver `MEMIF_BLOCK_INVALID` as a result.

2.4.2 Reference to MEMACC Address Area

The reference to a MEMACC Address Areas will be set for each partition via `/FeePartitionConfiguration/FeePartitionTarget` and not for each block. Every configured partition must reference the same MEMACC Address Area. As parallel FEE request processing is not supported and is not specified by AUTOSAR, referencing different Address Areas for each partition is currently not supported.

2.5 Flash Organization

The FEE implements different kinds of structures that are organized in a hierarchical manner to manage the assigned flash memory. The following structures are used:

- > Block Instance: Contains a single version of the data that was written for a configured block.
- > Chunk: Contains multiple block instances of the same block to speed up the search for the latest valid block instance.
- > Logical Sectors: Contains multiple chunks of different blocks.
- > Partition: Consists of a configured number of logical sectors.

The structure of a single partition is shown by Figure 2.1.

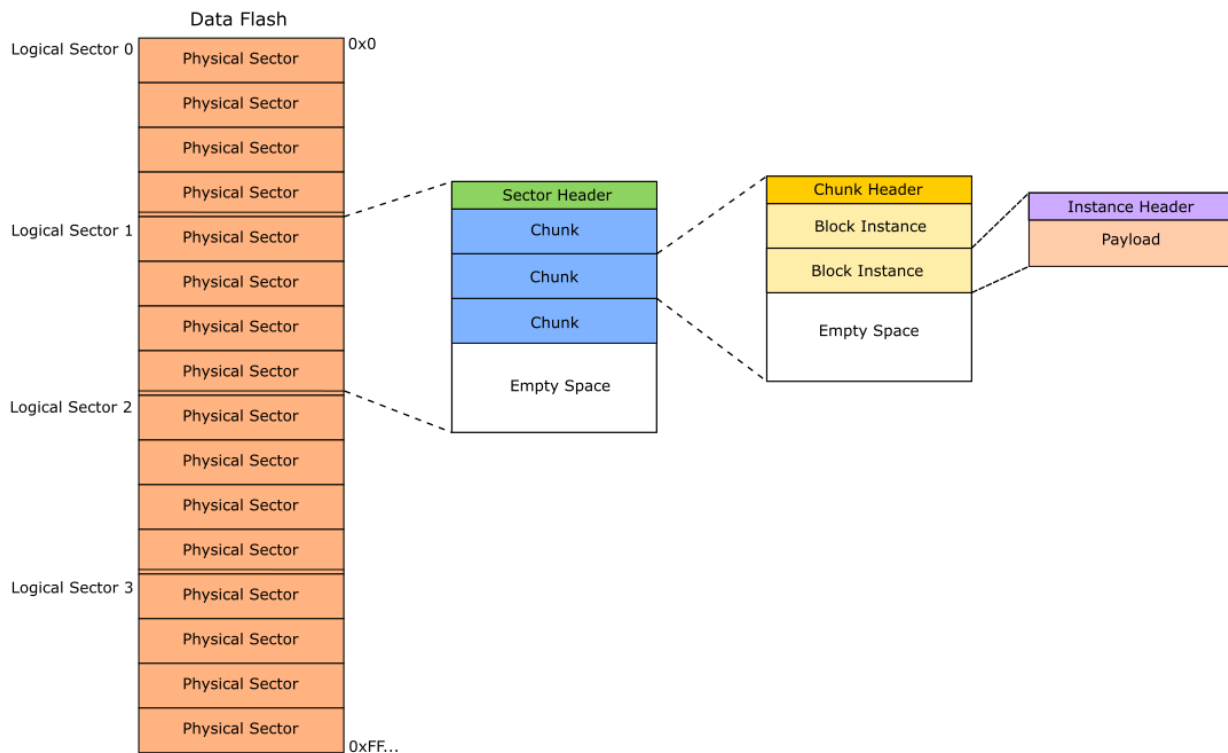


Figure 2.1 Hierarchical flash layout of a single partition

2.5.1 Partitions

Partitions are used to create different memory regions that are managed separately and operate independent of each other:

- > Errors in one partition do not affect data in other partitions.
- > Different management layout types can be used for different partitions.
- > Each partition defines its own alignment values and erased value.
- > Each partition defines its own set of logical sectors the partition is made of.



Note

Although partitions operate independent from each other, the FEE is only able to process one job at a time, even if the target blocks belong to different partitions.

2.5.2 Logical Sectors

A logical sector consists of at least one but typically many physical flash sectors that are adjacent to each other. It acts as the smallest erasable structure in the FEE.



Note

It is recommended to align the logical sectors with the physical sector boundaries. Although it is technically possible to start and end a logical sector within a physical sector, it would lead to wasting the rest of the affected physical sector space because it gets unusable for other data. The reason for this is, that the whole physical sector gets erased in case the FEE decides to erase the logical sector.



Basic Knowledge

The FEE requires at least two logical sectors that are placed in different physical flash sectors to function correctly. The reason for this is the garbage collection, that needs at least two independently erasable areas to keep the most recent data of the configured blocks and simultaneously free up new memory space.

2.5.2.1 Effects of Logical Sector Count

The FEE allows to configure between 2 and 32 logical sectors per partition. The amount of configured logical sectors have different effects on the FEE that can't be exactly quantified as the impact highly depends on the rest of the configuration and the used flash device and its properties. The following rules apply (assuming a fixed memory size):

- > A small logical sector count leads to large logical sectors. The frequency of garbage collection is low, but there is more memory wasted because more blocks are copied.
- > A large logical sector count leads to small logical sectors. The frequency of garbage collections is high but less blocks need to be copied which improves the lifetime of the flash.



Note

Because the effect of different logical sector counts depends on multiple configuration factors and properties, by default a logical sector count of 4 is recommended.

2.5.3 Layouts

The FEE implements different management flash layouts to optimize the data management for different flash devices.

2.5.3.1 Secure Layout

The secure layout adds additional static markers to the metadata of the flash structures that are independently written during the allocation process. These markers can then be used to evaluate the state of the allocation process to detect incomplete write operations (e.g., due to resets or cancel operations) and derive counter measures. The following markers are used:

- > Start Marker – Marks the beginning of an allocation process.
- > Seal Marker – Marks the end of the payload allocation.
- > Commit Marker – Marks the successful write of the seal marker.



Expert Knowledge

Chunks don't implement a seal marker to reduce the memory consumption of the management data. Due to the hierarchical relation of the structures (as shown in Figure 2.1. Hierarchical flash layout of a single partition), the commit marker of the first block instance within the chunk is used instead.



Caution

Due to the additional management data and flash write operations, the secure layout requires more resources and leads to a reduced performance of the FEE in terms of main function cycles that it takes to complete a job.

Using the secure layout is recommended by default to ensure robustness against data loss. Depending on the used flash device, other layout options may be used.

2.5.3.2 Slim Layout

The slim layout has basically the same structure as the secure layout (see Section 2.5.3.1) but with less static markers. This reduces the amount of flash consumed by the management data and improves the performance in terms of main function cycles required to execute a job.

The only static marker used is the commit marker, that is written after the rest of the structure that is protected by the commit marker was successfully written to flash.



Caution

Using the slim layout is only recommended in case the improved resource consumption and runtime performance is required and the used flash device is suitable.

A flash device is suitable in case data that was written by an incomplete flash write job doesn't change over time (i.e., the stored values in flash doesn't change if they are read multiple times over a longer period). Different values read from the same flash address might lead to data loss.

2.6 Lookup Table

The FEE implements a lookup table that stores a link to the latest chunk for each configured block in the RAM. This reduces the search effort for the latest valid instance because the location of the latest chunk is known. Key facts:

- > Each partition has its own separate lookup table.
- > If no lookup table is available or an error is detected, the lookup table is recovered by scanning all chunks in each valid sector.



Note

Each link in the lookup table requires 4-byte memory. Therefore, the total amount of RAM required depends on the number of configured blocks. See Section 2.2.1 for details about the structure of a link.

2.6.1 Persist Lookup Table

The persisting of the lookup table is automatically done by the FEE itself according to some metric. This is done to speed up the startup process. The API `Fee_30_FlexNor_PersistLookupTable()` is just a dummy for backward compatibility.



Note

Each partition requires a configured lookup table block that can be used to store the current version of the lookup table in. When using DaVinci Configurator, these blocks are created automatically for each partition.



Expert Knowledge

The lookup table persistency service uses the same mechanism to write the lookup table to flash that is used to write user blocks. Therefore, persisting the lookup table requires the same amount of time and management data as user blocks.

2.6.2 Load Lookup Table

In order to get the persisted lookup table for the startup, it must be loaded from the flash memory. This happens automatically without calling any explicit load API when the first job of the partition arrives.



Note

The loading of the lookup table happens for every partition separately, when the first job for the corresponding partition is requested.

To speed up the search for the latest lookup table chunk the FEE uses a link stored in each sector header to find the most recent instance of the lookup table. The content of the instance will then be copied to the lookup table in RAM and is then ready to be used.



Expert Knowledge

The FEE does not know whether the lookup table data loaded from flash memory displays the most recent addresses of all blocks. For example, in case of a reset before the persist operation could be started an old version of the lookup table might be read. In this case the FEE will perform a partial lookup table recovery to get the current flash state.

If the FEE

- > cannot load a lookup table from the flash memory, or
- > the loaded lookup table was persisted by a FEE in a version below 4.01.00,

a full recovery is performed during startup. For this reason, the first startup after an update can take longer than before. To reduce the startup time of all subsequent startups, the FEE persists the lookup table in the context of the next `Fee_30_FlexNor_Write()`, `Fee_30_FlexNor_InvalidatBlock()`, or `Fee_30_FlexNor_EraseImmediateBlock()` job.

2.6.3 Recover Lookup Table

Since the algorithms of the FEE rely on a lookup table in RAM, it must always be completely valid. In order to have a valid lookup table in RAM after the start up, two scenarios are possible:

- > In case the lookup table could be loaded, a partial recovery is performed, meaning that only flash memory not reflected by the lookup table must be scanned.
- > In case the lookup table could not be loaded successfully or was written by an older FEE implementation, the FEE will start a recover operation automatically to build up the lookup table from scratch.



Caution

For a full lookup table recovery the whole flash memory of the partition must be scanned, which slows down the performance of the startup. To avoid this, the FEE does persist the lookup table at regular intervals (according to a metric). During the next start up only a partial recovery has to be performed instead of a full recovery.



Expert Knowledge

In case the lookup table could be loaded from flash but the discrepancy between the stored links and the current flash image exceeds a certain threshold, still a full lookup table recovery is performed.

2.7 Garbage Collection

Each time a block is written, the FEE writes a new block instance to flash. This makes the last block instance automatically obsolete, but it remains in flash. To get rid of the obsolete data, free up new space and ensure an even wear of the flash memory, the FEE implements a garbage collection mechanism.

The garbage collection is triggered in case all but one logical sector are full and new data for a block shall be written. The process consists of multiple steps:

1. The FEE selects a source sector, that shall be garbage collected, from the full sectors. The decision which sector to select is based on a metric that is evaluated for each sector and consists of two parts:
 - > Amount of valid data that needs to be copied to the new sector. Minimizes the amount of data to copy to speed up the garbage collection process.
 - > Amount of erase cycles of the sector in relation to the other sectors. Ensures an even distribution of erases to reach an even wear of the flash memory.
2. After the source sector is selected, the FEE erases the target sector and sets up a new sector header.
3. The FEE copies all valid data from the source sector to the target sector.
4. The FEE marks the source sector for erase by writing an erase marker.



Note

Besides the purpose to free up space, a garbage collection can also be triggered internally as a recovery garbage collection (i.e. sector recovery) when damaged data is detected that might result in data loss in order to rescue the data. In both cases garbage collections will be only executed as a part of the following jobs: `Fee_30_FlexNor_Write()`, `Fee_30_FlexNor_InvalidateBlock()`, `Fee_30_FlexNor_EraseImmediateBlock()`.



Expert Knowledge

In case the garbage collection is interrupted by a reset or a cancel operation, the process is automatically resumed during the next write related job.

2.8 Block Configuration Update

The FEE allows to update its FEE block configuration. A block can be added or removed from a partition and its payload size can be changed.

**Caution**

Existing data of blocks, which were removed or whose payload size was changed, will be lost and removed from flash during the garbage collection process. Trying to read such a block will lead to job result `MEMIF_BLOCK_INVALID`, until the block has been rewritten according to the new configuration.

2.9 Partition Configuration Update

It is not possible to change the configuration of a partition for an existing FLASH image.

**Caution**

The FEE does not support a partition configuration update.

2.10 Cancellation

The FEE provides a cancel API that can be used to cancel ongoing asynchronous jobs. In many cases calling this function also leads to the cancellation of the flash job that is currently being executed.

Cancelling flash write jobs of FEE metadata can have expensive consequences (i.e., error recovery actions) in the following cases:

- > Allocating a new sector
- > Writing a chunk header
- > Writing an instance header

**Caution**

To avoid expensive error recovery, like the recovery of a sector, reallocation of a chunk or the execution of a garbage collection, the described actions are protected against cancellation.

If the FEE executes one of these actions and a cancellation of the job is requested, the FEE switches its state to `MEMIF_IDLE` and can accept a new job. Internally the FEE keeps processing the previous job until the protected actions are completed and cancels the rest of the job. If a new job is requested, it is stored, and the FEE switches its state to `MEMIF_BUSY`. As soon as the previous job was cancelled, the stored job is processed automatically.

2.11 Bootloader Mode

The FEE supports operation in application mode and operation in bootloader mode. One FEE instance for application and one instance for bootloader mode is required.

In application mode (the more common use case) the FEE is in full configuration mode. That means that all blocks that are persisted in flash are known to the FEE configuration and the full range of services is supported.

**Caution**

The application configuration must also include the bootloader blocks. Otherwise, these blocks would be considered as removed blocks and their data will be deleted in case of a garbage collection of this sector.

In bootloader mode FEE operates on a reduced partition configuration. That means of all the blocks that are persisted in flash, only the bootloader blocks are known to FEE configuration. To use the FEE in bootloader mode the configuration parameter `FeeGeneral/FeeBootloaderMode` must be set to `True`. In this mode the FEE provides services to read and write the configured blocks as usual but also has a few restrictions regarding other services, as described in the caution box below.

**Caution**

Restrictions:

- > The lookup table is not persisted in bootloader mode and hence the service to explicitly trigger the lookup table persistency is disabled. If called in bootloader mode, the service will return `E_NOT_OK`.
- > The size of the lookup table that is usually determined automatically must be configured by the user for each partition. See Section 5.2.2 for further information on the configuration of this parameter.

**Expert Knowledge**

It is possible to use the FEE within bootloader context without using the bootloader mode. In that case, the bootloader blocks must be in their own partition. That has the advantage that all services can be used without restrictions and the disadvantage that potentially a lot of flash memory is needed.

2.12 Suspend and Resume Internal Recovery Jobs

The FEE offers the possibility to suspend internal recovery jobs. To avoid data loss caused by pages in an unsafe state, the FEE performs some recovery jobs as part of its internal jobs. Internal jobs are always performed, but recovery jobs only in case of write-like user jobs. Even though the recovery mechanisms are highly recommended to be activated, they do consume processing cycles. To suspend the recovery jobs, e.g. for time-critical periods like the startup, the service `Fee_30_FlexNor_SuspendRecoveryJobs()` of the FEE can be called. All other internal jobs like the startup of a partition or finishing a pending garbage collection are performed as usual as these are fundamental for ensuring the functionality of the FEE.

Conversely, afterwards, to enable the FEE to perform all internal recovery operations, the `Fee_30_FlexNor_ResumeRecoveryJobs()` service of the FEE has to be called.

By default, the FEE executes all pending internal jobs, so invoking the service `Fee_30_FlexNor_ResumeRecoveryJobs()` without calling the service `Fee_30_FlexNor_SuspendRecoveryJobs()` first has no effect on the behavior of the FEE.



Caution

It is not recommended at all to suspend internal recovery jobs of the FEE for longer than necessary in order to improve the execution time of single job requests. These recovery operations minimize the risk of data loss and hence increase the robustness of the FEE.

2.13 Suspend and Resume Write Related Jobs

The FEE supports suspension of lower layer write/erase job requests. To block the FEE to request write/erase jobs from the lower layer, the `Fee_30_FlexNor_SuspendFlashWrites()` service of the FEE must be called. Conversely, afterwards, to enable the FEE requesting these job requests from the lower layer, the `Fee_30_FlexNor_ResumeFlashWrites()` service of the FEE has to be called.

By default, the FEE is allowed to request any kind of job requests from the lower layer, so invoking the service `Fee_30_FlexNor_ResumeFlashWrites()` without calling the service `Fee_30_FlexNor_SuspendFlashWrites()` first has no effect on the behavior of the FEE.

Once the service `Fee_30_FlexNor_SuspendFlashWrites()` has been called an ongoing write/erase job of the lower layer is not affected and finished before any further write related jobs are suspended - any read requests are not affected by this call. If the FEE has to request a write/erase job from the lower layer to proceed with its current job, its status will be `MEMIF_BUSY` until the service `Fee_30_FlexNor_ResumeFlashWrites()` is invoked and the current job can be finished.

2.14 GetEraseCycleCounter

The FEE provides the API `Fee_30_FlexNor_GetEraseCycleCounter()` to obtain the maximal erase cycle counter (ECC) of all sectors to a provided partition. This API is only available after the partition was already started up, i.e. one of the following jobs was performed beforehand:

```
> Fee_30_FlexNor_Read()
> Fee_30_FlexNor_Write()
> Fee_30_FlexNor_InvalidateBlock()
> Fee_30_FlexNor_EraseImmediate()
> Fee_30_FlexNor_PersistLookupTable()
```

This is important to ensure that the FEE reads the necessary information from the flash into the RAM.



Expert Knowledge

In fact all ECCs of one partition are expected to have nearly the same value because of the FEE internal Garbage Collection behavior.

2.15 Exception Handling

It is mandatory that the integration of the FEE and the hardware platform fulfill the requirements listed in Section 3.13.

If these requirements are not met, the FEE is not able to work properly. However, if one requirement is not met, the effect will not be detectable immediately. Hence, it is quite difficult to recognize wrong integrations. Usually, they are recognized when it is already too late.

To support the integrator, the FEE tries to detect unexpected effects which are usually caused by integration errors and can lead to data loss or unexpected behavior. It is very important that the source of the exception, i.e. the wrong integration, is fixed. When the exception reporting mechanism is enabled, the FEE will notify the user in case an exception is detected by invoking a callback which is provided by the integrator.

This callback provides information for the integrator/user, what must be fixed. It is highly recommended to keep this setting enabled.

Additionally, the FEE can provide debug information to analyze the root cause of an exception. If this feature is enabled, the callback additionally notifies the user about every milestone which is reached during the FEE job processing.

2.15.1 Milestones and Exceptions

The DiagnosticHandler unit within the FEE is responsible for managing exception handling and job progress tracking. It performs two key functions:

- > **Error Notification:** When an exception occurs, the DiagnosticHandler stores the error and, if configured, notifies the user about the issue.
- > **Milestone Notification:** During job execution, the unit monitors progress using predefined milestones. If milestone notifications are enabled, the user is informed as the job reaches each milestone.

In both cases the same callback defined in Section 2.15.2.1 is used for notifications with the following parameters set:

- > The severity in `publishedInfo->diagnosticInformation.severity` of type `Fee_30_FlexNor_SeverityType`.
- > The latest reached milestone in `publishedInfo->diagnosticInformation.currentMilestone` of type `Fee_30_FlexNor_MileStoneIdType`.

- > Information about the currently running user job
`publishedInfo->currentJobInformation` of type
`Fee_30_FlexNor_CurrentJobInformationType`.

2.15.1.1 Milestones

The progress of a FEE job is tracked by so-called **milestones**. Every FEE job consists of several job steps. When an important job step is started or finished, the currently active FEE unit informs the DiagnosticHandler that a milestone has been reached. The DiagnosticHandler stores the latest reached milestone. If debug notifications are enabled, the user is informed when a milestone is reached. This sequence is shown in Figure 2.2.

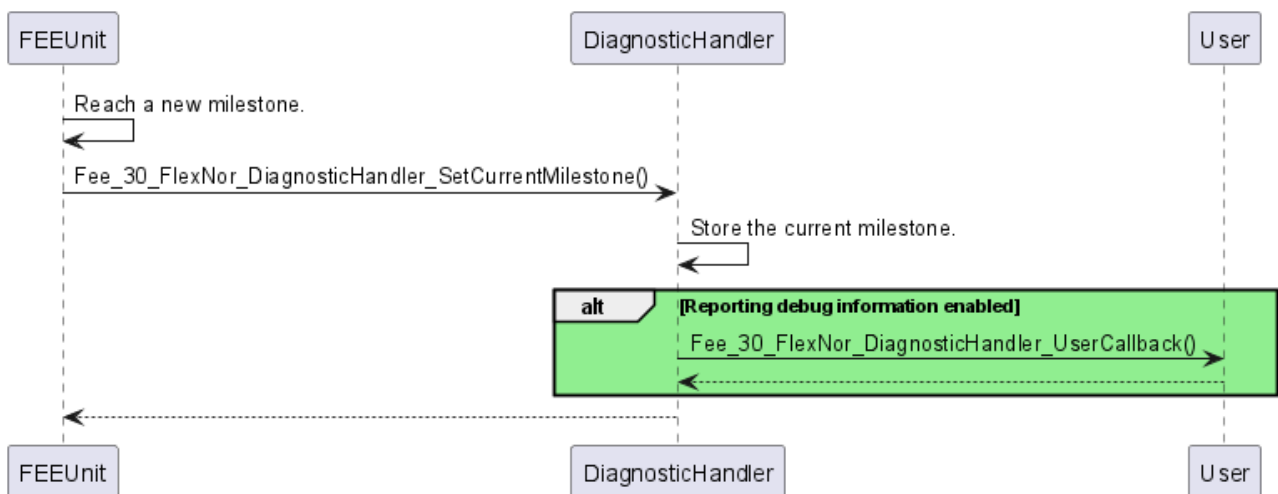


Figure 2.2 Notification sequence in case a milestone was reached.

When the user is notified about a newly reached milestone, the following conditions apply:

- > The severity `publishedInfo->diagnosticInformation.severity` is set to `FEE_30_FLEXNOR_DIAGSEVERITY_INFORMATION`.
- > The diagnostic ID `publishedInfo->diagnosticInformation.diagnosticId` is set to `FEE_30_FLEXNOR_DIAGID_NO_ERROR`.

2.15.1.2 Errors and Warnings

If any unit of the FEE detects an issue which gives hint to an integration error, the unit raises either an error or a warning to the DiagnosticHandler, depending on the severity of the issue. The DiagnosticHandler stores the risen error or warning and informs the user if configured. This sequence is shown in Figure 2.3.

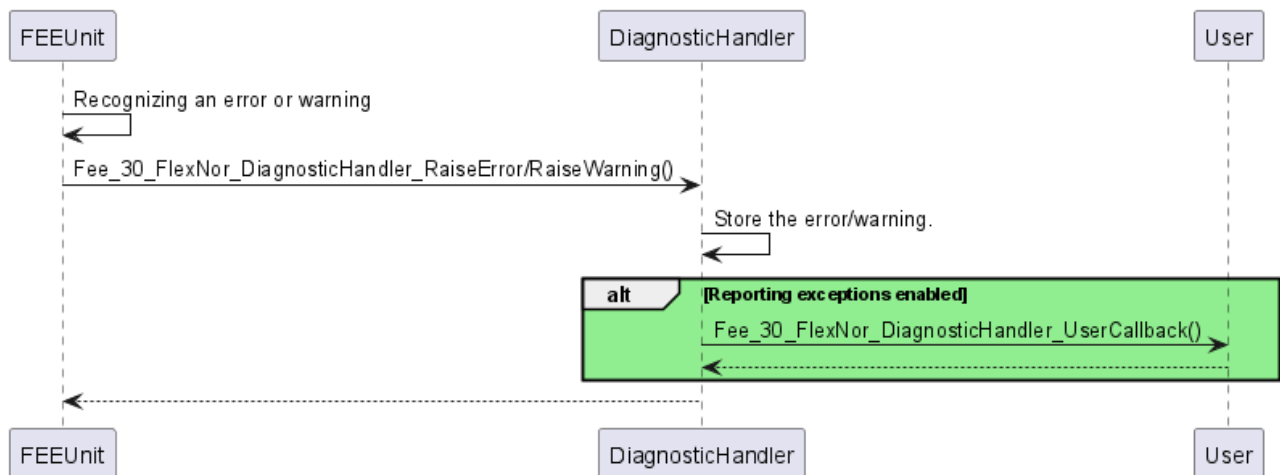


Figure 2.3 Notification sequence in case an error or a warning occurred.

When the user is notified about an error or warning, the callback parameter includes:

- > The diagnostic ID of the error or warning in
`publishedInfo->diagnosticInformation.diagnosticId` of type
`Fee_30_FlexNor_DiagnosticIdType`.

2.15.2 Integration

2.15.2.1 Callback Implementation

The integrator is responsible for implementing the user callback

```
void Fee_30_FlexNor_DiagnosticHandler_UserCallback(
    Fee_30_FlexNor_EventPublishedInfoPtrType publishedInfo);
```

The interface definition of the user callback is provided via

```
Fee_30_FlexNor_DiagnosticHandler_Cbk.h.
```

All relevant data types, including the list of possible milestones that can be reached and errors or warnings which can be raised and reported by the FEE, are defined in the module header file: `Fee_30_FlexNor_Types.h`.

The parameter `publishedInfo` gives information about the exception or the current milestone. It is described in Section 2.15.1.

The callback must be provided if either exception reporting or debug notification is enabled, as explained in Section 2.15.2.2. Only if both are disabled, it is not necessary to provide this callback.

An example implementation for the callback is provided in Section 2.15.3.

2.15.2.2 Configuration

Error detection is always enabled, but the reporting of errors and warnings must be enabled via the configuration parameter

```
/MICROSAR/Fee_30_FlexNor/Fee/FeeGeneral/FeeReportExceptions.
```

It is highly recommended to keep the reporting of errors and warnings enabled.

The debug notifications can be enabled via the configuration parameter

```
/MICROSAR/Fee_30_FlexNor/Fee/FeeGeneral/FeeReportDebugNotifications.
```

It is recommended to keep this option disabled and enable it, when a debug scenario occurs.

2.15.3 Example Implementation of the User Callback

The following example implementation of the user callback can be extended with code to log the information provided via the function argument.

```
#include "Fee_30_FlexNor_DiagnosticHandler_Cbk.h"

void Fee_30_FlexNor_DiagnosticHandler_UserCallback(
    Fee_30_FlexNor_EventPublishedInfoPtrType publishedInfo)
{
    /* Get the diagnostic information and the user job. */
    Fee_30_FlexNor_DiagnosticInformation diagInfo = publishedInfo->
↪diagnosticInformation;
    Fee_30_FlexNor_CurrentJobInformationType job = publishedInfo->
↪currentJobInformation;

    /* Get the numerical values of the diagnostic information. */
    Fee_30_FlexNor_MileStoneIdType currentMilestone = diagInfo.currentMilestone;
    Fee_30_FlexNor_DiagnosticIdType diagnosticId = diagInfo.diagnosticId;
    Fee_30_FlexNor_SeverityType severity = diagInfo.severity;

    /* Get the user job information. */
    Fee_30_FlexNor_ServiceId serviceId = job.serviceId;
    Fee_30_FlexNor_BlockIdType blockId = job.blockId;
    Fee_30_FlexNor_PartitionIdType partitionId = job.partitionId;

    /*
     * Add code here to log the following variables:
     *
     * - severity
     * - diagnosticId
     * - currentMilestone
     * - partitionId
     * - blockId
     * - serviceId
     */
}
```

3 Integration

3.1 Embedded Implementation

The module contains these source code files. Some files are generated by the configuration tool DaVinci Configurator Classic.

**Note**

All internal files listed below are not supposed to be included by anyone else other than the FEE itself. Only Fee_30_FlexNor.h shall be included directly by other components.

File Name	Description	Integration Tasks
Fee_30_FlexNor.h	Defines the public interface of the FEE module.	-
Fee_30_FlexNor_Cbk.h	Defines the interface for the FLS and MEMACC callbacks.	-
Fee_30_FlexNor_ErrorChecks.h	Internal header file relevant for providing the interfaces for Error/DET checks.	-
Fee_30_FlexNor_Initializer.h	Internal header file relevant for providing the interfaces for initialization of the component's units.	-
Fee_30_FlexNor_Types.h	Internal header file relevant for types used by the FEE.	-
Fee_30_FlexNor_<*>Factory.h	Internal header file responsible for providing the interface for the respective unit creation.	-
Fee_30_FlexNor_<*>Internal.h	Internal header file responsible for providing respective internal unit interfaces.	-
Fee_30_FlexNor_<*>Machine.h	Internal header file responsible for providing the interfaces the state machine functionality of the respective unit.	-
Fee_30_FlexNor_Chunk.h	Internal header file responsible for providing general chunk interfaces.	-
Fee_30_FlexNor_SecureChunk.h	Internal header file responsible for providing secure chunk interfaces.	-
Fee_30_FlexNor_SlimChunk.h	Internal header file responsible for providing slim chunk interfaces.	-

continues on next page

Table 3.1 – continued from previous page

Fee_30_FlexNor_Instance.h	Internal header file responsible for providing generic instance interfaces.	-
Fee_30_FlexNor_SecureInstance.h	Internal header file responsible for providing secure instance interfaces.	-
Fee_30_FlexNor_SlimInstance.h	Internal header file responsible for providing slim instance interfaces.	-
Fee_30_FlexNor_Sector.h	Internal header file responsible for providing sector interfaces.	-
Fee_30_FlexNor_FlashAccess.h	Internal header file responsible for providing interfaces for the flash access.	-
Fee_30_FlexNor_ChunkSearch.h	Internal header file responsible for providing the chunk search functionality.	-
Fee_30_FlexNor_CopyBlock.h	Internal header file responsible for providing the copy block functionality.	-
Fee_30_FlexNor_GarbageCollection.h	Internal header file responsible for providing the garbage collection functionality.	-
Fee_30_FlexNor_InternalJobs.h	Internal header file responsible for providing the internal job handling.	-
Fee_30_FlexNor_LookUpTable.h	Internal header file responsible for providing the lookup table functionality.	-
Fee_30_FlexNor_Partition.h	Internal header file responsible for providing partition interfaces and high-level logic for data management.	-
Fee_30_FlexNor_SectorContainer.h	Internal header file responsible for providing access to sector objects for partitions.	-
Fee_30_FlexNor_Startup.h	Internal header file responsible for providing the startup functionality.	-
Fee_30_FlexNor_ConfigInterface.h	Internal header file responsible for providing the configuration abstraction interfaces.	-
Fee_30_FlexNor_Scheduler.h	Internal header file responsible for providing the scheduler functionality.	-
Fee_30_FlexNor_Shared.h	Internal header file responsible for providing interfaces for shared functionality between the units.	-

continues on next page

Table 3.1 – continued from previous page

Fee_30_FlexNor_DiagnosticHandler.h	Internal header file responsible for providing the interface of the diagnostic handler.	-
Fee_30_FlexNor_DiagnosticHandler_Cbk.h	Defines the interface for the user callback of the diagnostic handler.	-
Fee_30_FlexNor.c	Implements the FEE module's API.	-
Fee_30_FlexNor_ErrorChecks.c	Implements the Error/DET checks.	-
Fee_30_FlexNor_Initializer.c	Implements the initialization for units.	-
Fee_30_FlexNor_<*>Factory.c	Implements the creation of runtime objects of flash structures.	-
Fee_30_FlexNor_<*>Machine.c	Implements the unit's state machine.	-
Fee_30_FlexNor_Chunk.c	Implements the generic chunk functionality.	-
Fee_30_FlexNor_SecureChunk.c	Implements the secure chunk functionality.	-
Fee_30_FlexNor_SlimChunk.c	Implements the slim chunk functionality.	-
Fee_30_FlexNor_Instance.c	Implements the generic instance functionality.	-
Fee_30_FlexNor_SecureInstance.c	Implements the secure instance functionality.	-
Fee_30_FlexNor_SlimInstance.c	Implements the slim instance functionality.	-
Fee_30_FlexNor_Sector.c	Implements the sector functionality.	-
Fee_30_FlexNor_FlashAccess.c	Implements the flash access functionality.	-
Fee_30_FlexNor_ChunkSearch.c	Implements the chunk search functionality.	-
Fee_30_FlexNor_CopyBlock.c	Implements the copy block functionality.	-
Fee_30_FlexNor_GarbageCollection.c	Implements the garbage collection functionality.	-
Fee_30_FlexNor_InternalJobs.c	Implements the internal jobs handling.	-
Fee_30_FlexNor_LookUpTable.c	Implements the lookup table functionality.	-
Fee_30_FlexNor_Partition.c	Implements the partition functionality.	-
Fee_30_FlexNor_SectorContainer.c	Implements a managed access to sector objects within RAM.	-

continues on next page

Table 3.1 – continued from previous page

Fee_30_FlexNor_Startup.c	Implements the startup functionality.	-
Fee_30_FlexNor_ConfigInterface.c	Implements the config abstraction.	-
Fee_30_FlexNor_Scheduler.c	Implements the scheduler unit.	-
Fee_30_FlexNor_Shared.c	Implements the shared functionality between units.	-
Fee_30_FlexNor_DiagnosticHandler.c	Implements the diagnostic handler.	-
Fee_30_FlexNor_Cfg.h	Contains the static configuration of this module.	-
Fee_30_FlexNor_MemoryInterface.h	Contains the interfaces to the underlying FLS driver or MEMACC driver.	-
Fee_30_FlexNor_Lcfg.c	Contains the link-time part of the configuration.	-
Fee_30_FlexNor_MemMap.h	Generated header file that contains the memory mapping sections of the FEE module.	-

Table 3.1 Source Code Files :header-rows: 1 :widths: 20 60 20

3.2 Initialization

The FEE is initialized after `Fee_30_FlexNor_Init()` has been called.



Caution

Before requesting a job, the FEE must be initialized first. Otherwise, all job requests are rejected by default.

The initialization just prepares the internal state of the FEE and enables it to accept jobs. To be able to process jobs the FEE must

- > read all sector headers from flash
- > and load the latest version of the lookup table (if available)

to discover the current state of the data kept in flash. These actions are executed in the context of the first job that is being processed.

3.3 Main Functions

Due to the asynchronous processing of jobs, the FEE requires a module that calls its `Fee_30_FlexNor_MainFunction()`. This can either be an operating system or any other module that is interested in FEE jobs being executed.

However the FEE `MainFunction` shall not be interrupted by API service calls from upper layers (e.g. NVM, FBL, ...). Also calls to API services shall not be interrupted by the

MainFunction.

**Caution**

The FEE MainFunction must be executed in the same process context as upper layer modules using API services from FEE directly (e.g. NVM MainFunction, FBL).

3.4 Callouts

3.4.1 Lower Layer Interaction

The FEE offers the usage of callback notification functions for the underlying driver to inform the FEE that a job has finished successfully or not.

FLS

The `Fee_30_FlexNor_JobEndNotification()` is called when a job is completed with a positive result and the `Fee_30_FlexNor_JobErrorNotification()` is called when a job is cancelled, aborted or failed.

MEMACC

The `Fee_30_FlexNor_AddressAreaJobEndNotification()` is called when a job is completed with a positive result, cancelled, aborted or failed.

**Note**

The interaction between the FEE and the underlying driver does not need to be performed via a notification mechanism. Polling mode can also be chosen if desired. Using notifications is the preferred method however, since the `Fee_30_FlexNor_MainFunction()` does not need to be polled as often, thus decreasing CPU time.

3.4.2 Upper Layer Interaction

3.4.2.1 NvM

The NVM offers the usage of callback notification functions for the FEE, to get informed that a job has finished successfully or not. The job end notification is called in case a job was completed successfully. The job error notification is called in case a job failed or the job was canceled.

**Note**

The callback notification to the NvM is used if the polling mode at the NvM is disabled and valid callback function names and the correct NvM include file, that provides the callback prototypes, are configured.

3.4.2.2 User

The integrator can offer an diagnostic callback function for the Fee. Depending on the configuration, this callback is invoked, if an issue is detected which can lead to a malfunction and suggests that the Fee was not integrated correctly. See *Exception Handling*.

3.5 Synchronization

3.5.1 Critical Sections

Currently the FEE does not use critical sections.

3.6 Memory Sections

In case MEMACC is used instead of a FLS it is an integration task to align the `Fee_30_FlexNor_InternalBuffer` placed within `Fee_30_FlexNor_Lcfg.c` accordingly.

The alignment can be determined via `/FeeGeneral/FeeBufferAlignmentValue` reference pointing to the Address Area where the alignment value is noted:

```
/MemAccAddressAreaConfiguration/MemAccBufferAlignmentValue.
```

3.7 MemMap

Delivery with MICROSAR Release <R27:

`Fee_30_FlexNor_MemMap.h` is available as a static file. Memory mapping sections of the FEE module are included within the global `MemMap.h` file (via `Fee_30_FlexNor_MemMap.inc`), which will be included by `Fee_30_FlexNor_MemMap.h`.

Delivery with MICROSAR Release >=27:

`Fee_30_FlexNor_MemMap.h` is available as a dynamic file. This header file is generated by an own component “MemMap” (for more information refer to [4]). Memory mapping sections of the FEE module are directly included via the generated `Fee_30_FlexNor_MemMap.h` file.

3.8 Module Det

The usage of the DET is optional and records all development errors for diagnostic purposes. Its usage can be enabled and disabled by the switch `FEE_30_FLEXNOR_DEV_ERROR_DETECT`.

3.9 Module Fls

The FLS driver [5] provides the access to the underlying hardware. The specific properties of the flash hardware influence the configuration of the FEE component. Its services are called to request a special job to the driver. The synchronous cancelling of FLS is supported by FEE component.

3.10 Module MEMACC

The MEMACC [6] provides the access to the underlying hardware. The specific properties of the flash hardware influence the configuration of the FEE component. Its services are called to request a special job to the MEMACC. The asynchronous cancelling of MEMACC is supported by FEE component.

3.11 Dependencies on HW Modules

The FEE is principally hardware independent. Nevertheless, the FLS driver has to provide some parameters (defined in the BSWMD file) the FEE relies on. Examples for this are the page size or sector sizes.

3.12 Error Reporting Det

The module has these properties used for the error reporting to the Det.

- > FEE_MODULE_ID = 21
- > FEE_VENDOR_ID = 30

3.13 Requirements and Recommendations

This section shall point out requirements the user system needs to comply with when using this FEE module. Additionally, recommendations are made for this module and its configuration regarding limitations of FEE. If at least one requirement is not met, the FEE will be out of specification, which means that the behavior might not be as specified. Please ensure that all the following requirements are fulfilled before using FEE.

3.13.1 Requirements for the User System

- > The system shall try to reduce the number of unintended resets as far as possible. Although the FEE can cope with single resets, it is the responsibility of the user system to avoid unintended resets (e.g. by undervoltage management).
- > Especially, at the shutdown the system shall wait until the FEE is idle again before cutting off the power.
- > The `Fee_30_FlexNor_MainFunction()` shall not be reentered. (Attention with preemptive tasks).
- > No other user shall access the flash memory without synchronization (e.g. via VMEMACCM or MEMACC module).
- > The user shall compute the write cycles for every block and shall check that the flash will be inside the number of erase cycles specified by the hardware supplier over the whole lifetime of the ECU.

**Caution**

To ensure that the Fee works reliably, the endurance **must** be calculated. This calculation **must** be made by the Endurance Simulation, which is provided by an internally developed web interface. Please contact the support for performing this simulation.

3.13.2 Requirements for the Flash Memory and the Driver

Following requirements apply for the FLS as well as the MEMACC module in combination with the MEM.

3.13.2.1 General Requirements

Only addressed flash pages and sectors shall be affected during a job.

3.13.2.2 Write-Related

- > Only if intended data was written successfully and persistently to flash memory, FLS driver shall return with `MEMIF_JOB_OK` or MEMACC shall return with `MEMACC_OK`.
- > A flash page that is read as erased can be written with any arbitrary value.
- > Successfully written flash cells shall not alter their values over time.
- > Write aborts (e.g. due to reset) shall only affect the flash page which is currently being written.

3.13.2.3 Read-Related

- > Only if intended data was read successfully from flash memory, FLS driver shall return with `MEMIF_JOB_OK` or MEMACC shall return with `MEMACC_OK`.
- > If during a FLS read operation a ECC error is detected in one of the addressed pages, `MEMIF_JOB_FAILED` shall be reported to FEE. If during a MEMACC read operation a ECC error is detected in one of the addressed pages, `MEMACC_FAILED`, `MEMACC_ECC_CORRECTED` or `MEMACC_ECC_UNCORRECTED` shall be reported to FEE.
- > A FLS or MEMACC read operation shall not modify the flash memory.

3.13.2.4 Erase-Related

- > Only if intended flash area was erased successfully and persistently, FLS driver shall return with `MEMIF_JOB_OK` or MEMACC shall return with `MEMACC_OK`.
- > Erased flash cells shall remain erased over time until a write job is performed.
- > Erase aborts (e.g. due to reset) shall only affect the flash sector which is currently being erased.

3.13.2.5 Recommendations

- > Assure that your block configuration does not need more than 30 percent of configured memory size.
- > Double check that the alignment settings were selected correctly for your hardware (Use hardware manuals).

3.13.3 Update Information

Observe the following points when updating to a newer Fee_30_FlexNor version:

- > An update from another (legacy) Fee component to the Fee_30_FlexNor is in principle possible.



Caution

The memory layouts (and hence the components themselves) are **not** compatible and hence already written data is lost after an update.

- > The version of the Bootloader Fee and the one of the used Application Fee must be compatible.

4 API Description

4.1 Provided Functions

The module provides these functions.

Service functions are implemented by the module and are typically called by upper-layer modules. They are declared in Fee_30_FlexNor.h.

Callback functions are implemented by the module and are typically called by lower-layer modules. They are declared in Fee_30_FlexNor_Cbk.h.

4.1.1 Fee_30_FlexNor_Init

Prototype
void Fee_30_FlexNor_Init ()
Return Codes
N/A
Functional Description
Initialization function Service to initialize the module Fee_30_FlexNor. It initializes all variables and sets the module state to initialized. The FEE module doesn't support any runtime configuration. Hence, a pointer to the configuration structure is not needed by this service. The FEE does not initialize the underlying FLS or MEMACC module, but this shall be done by the ECUM module. (This is no AUTOSAR deviation.)
Limitations
> This service shall not be called during a pending job.
Particularities
> This service is always available.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.1 Fee_30_FlexNor_Init

4.1.2 Fee_30_FlexNor_SetMode

Prototype	
void Fee_30_FlexNor_SetMode (MemIf_ModeType mode)	
Parameters	
mode [in]	Parameter will be ignored
Return Codes	
N/A	
Functional Description	

continues on next page

Table 4.2 – continued from previous page

Dummy service for setting the mode. Only implemented for compatibility reasons.
This function implements no functionality and will ignore the Mode parameter.

Options

callcontext: TASK
reentrant: FALSE
synchronous: TRUE

Table 4.2 Fee_30_FlexNor_SetMode

4.1.3 Fee_30_FlexNor_Read

Prototype

Std_ReturnType **Fee_30_FlexNor_Read**(Fee_30_FlexNor_BlockNumberType BlockNumber, Fee_30_FlexNor_BlockOffsetType BlockOffset, Fee_30_FlexNor_DataPtrType DataBufferPtr, Fee_30_FlexNor_LengthType Length)

Parameters

BlockNumber [in]	Unique identifier of block
BlockOffset [in]	Offset in the block
DataBufferPtr [in]	Pointer to buffer from upper layer
Length [in]	Amount of bytes which should be read

Return Codes

E_OK The job was accepted
E_NOT_OK The job was rejected

Functional Description

This function starts the read processing for the specified block.

**Info**

The job processing is asynchronous. The result of the finished job can be polled by calling `Fee_30_FlexNor_GetJobResult()`.

When the current block is found, the parameters BlockOffset and Length are used to call the Read function of the underlying FIs driver to read out the content of the flash memory to the provided DataBufferPtr. As the processing is asynchronous the return value of the FIs Read function can only be returned indirectly via the job result.

Additionally, if development mode is configured, parameter checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to Section 2.3.1).

**Info**

`Fee_30_FlexNor_Read()` might deliver old data, i.e. not the latest instance written, in case most recent instance is corrupt (e.g. due to resets). The application must cope with the possibility of getting old data.

continues on next page

Table 4.3 – continued from previous page

Particularities
> This service is always available.
Options
callcontext: TASK reentrant: FALSE synchronous: FALSE

Table 4.3 Fee_30_FlexNor_Read

4.1.4 Fee_30_FlexNor_Write

Prototype	
Std_ReturnType Fee_30_FlexNor_Write (Fee_30_FlexNor_BlockNumberType BlockNumber, Fee_30_FlexNor_ConstDataPtrType DataBufferPtr)	
Parameters	
BlockNumber [in]	Unique identifier of block
DataBufferPtr [in]	Pointer to buffer from upper layer
Return Codes	
E_OK The job was accepted E_NOT_OK The job was rejected	
Functional Description	
This function starts the write processing for the specified block.	
 Info The job processing is asynchronous. The result of the finished job can be polled by calling <code>Fee_30_FlexNor_GetJobResult()</code>.	
When the next free area for the block was found, the content of the provided <code>DataBufferPtr</code> is written to the flash memory. Additionally, management information is stored to identify the block. The real count of bytes which must be written depends on the hardware specific page alignment and the size of the management information. As the processing is asynchronous the return value of the FIs write function can only be returned indirectly via the job result. Additionally, if development mode is configured, parameter checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to Section 2.3.1).	
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.4 Fee_30_FlexNor_Write

4.1.5 Fee_30_FlexNor_InvalidateBlock

Prototype	
Std_ReturnType Fee_30_FlexNor_InvalidateBlock (Fee_30_FlexNor_BlockNumberType BlockNumber)	
Parameters	
BlockNumber [in]	Unique identifier of block that shall be invalidated
Return Codes	
E_OK The job was accepted E_NOT_OK The job was rejected	
Functional Description	
This service invokes the invalidation procedure for the selected block. If the service succeeds the most recent data block is marked as INVALID. When the last/current block was found the next free area will be written with special invalidate information. As this information is saved in flash memory it is resistant against resets. Additionally, if development mode is configured, parameter checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to chapter Section 2.3.1).	
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: FALSE	

Table 4.5 Fee_30_FlexNor_InvalidateBlock

4.1.6 Fee_30_FlexNor_EraseImmediateBlock

Prototype	
Std_ReturnType Fee_30_FlexNor_EraseImmediateBlock (Fee_30_FlexNor_BlockNumberType Block-Number)	
Parameters	
BlockNumber [in]	Unique identifier of block with immediate data that shall be erased
Return Codes	
E_OK The job was accepted E_NOT_OK The job was rejected	
Functional Description	

continues on next page

Table 4.6 – continued from previous page

This function doesn't erase flash memory.

The addressed block is marked as invalid, thus a subsequent read request on the invalidated block completes with `MEMIF_BLOCK_INVALID`.

Additionally, if development mode is configured, parameter checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to chapter Section 2.3.1).

**Note**

The FEE does not care about immediate blocks; there is no special handling. Thus this service does exactly the same as `Fee_30_FlexNor_InvalidateBlock()`

Particularities

> This service is always available.

Options

callcontext: TASK
reentrant: FALSE
synchronous: FALSE

Table 4.6 Fee_30_FlexNor_EraseImmediateBlock

4.1.7 Fee_30_FlexNor_PersistLookupTable

Prototype

Std_ReturnType **Fee_30_FlexNor_PersistLookupTable**(Fee_30_FlexNor_PartitionIdType PartitionId)

Parameters

PartitionId [in]	Dummy parameter.
------------------	------------------

Return Codes

E_OK: always

Functional Description

Dummy API for backward compatibility.

Particularities

> The functionality of persisting the LookupTable is automatically done by the Fee itself. The API is just a dummy for backward compatibility.

Options

callcontext: TASK
reentrant: TRUE
synchronous: TRUE

Table 4.7 Fee_30_FlexNor_PersistLookupTable

4.1.8 Fee_30_FlexNor_Cancel

Prototype
void Fee_30_FlexNor_Cancel ()
Functional Description
This service cancels a currently pending job. The state of the FEE will be set to <code>MEMIF_IDLE</code>. If the FEE is currently <code>IDLE</code>, calling this service is without any effect. Additionally, if development mode is configured, checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to Section 2.3.1).
Caution While this service is specified as asynchronous within [1], it is still synchronous (e.g. ready to accept new job immediately) due to compatibility reason for older AUTOSAR releases and therefore for the usage of a NvM expecting a synchronous FEE Cancel request.
Particularities
> This service is always available.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.8 Fee_30_FlexNor_Cancel

4.1.9 Fee_30_FlexNor_GetStatus

Prototype
MemIf_StatusType Fee_30_FlexNor_GetStatus ()
Return Codes
<code>MEMIF_UNINIT</code>: The FEE is currently not initialized -> <code>Fee_30_FlexNor_Init()</code> must be called to use the functionality of the FEE. <code>MEMIF_IDLE</code>: The FEE is currently idle -> no asynchronous job available. <code>MEMIF_BUSY</code>: The FEE is currently busy -> an asynchronous job is currently being processed by the FEE.
Functional Description
This service returns the FEE's current module state synchronously.
Info The <code>Fee_30_FlexNor</code> does not use <code>MEMIF_BUSY_INTERNAL</code>.
Particularities
> This service is always available.

continues on next page

Table 4.9 – continued from previous page

Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.9 Fee_30_FlexNor_GetStatus

4.1.10 Fee_30_FlexNor_GetJobResult

Prototype
MemIf_JobResultType Fee_30_FlexNor_GetJobResult()
Return Codes
Current module job result MEMIF_JOB_OK: The last job has been finished successfully. MEMIF_JOB_PENDING: The last job is waiting for execution or currently being executed. MEMIF_JOB_CANCELLED: The last job has been cancelled by the <code>Fee_30_FlexNor_Cancel()</code> service. MEMIF_JOB_FAILED: The Flash driver reported an error or the FEE could not achieve the requested job due to hardware errors (e.g. memory cell defects). MEMIF_BLOCK_INVALID: Either, the requested block has been invalidated previously by the services <code>Fee_30_FlexNor_InvalidateBlock()</code> / <code>Fee_30_FlexNor_EraseImmediateBlock()</code> . Or, the data of requested block could not be read, because the data is corrupt. Or, the block has never been written before and thus cannot be found.
Functional Description
This service returns the result of the last job executed.
Particularities
> This service is always available.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.10 Fee_30_FlexNor_GetJobResult

4.1.11 Fee_30_FlexNor_SuspendRecoveryJobs

Prototype
void Fee_30_FlexNor_SuspendRecoveryJobs()
Functional Description

continues on next page

Table 4.11 – continued from previous page

This service prevents the FEE from executing internal recovery jobs.	
	Caution Suspending internal recovery jobs of the FEE for longer than necessary is not recommended as this increases the chance of data loss.
	Info Only internal recovery jobs are affected. Other internal jobs like the startup of a partition or finishing pending garbage collections are executed as usual.
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.11 Fee_30_FlexNor_SuspendRecoveryJobs

4.1.12 Fee_30_FlexNor_ResumeRecoveryJobs

Prototype	
void Fee_30_FlexNor_ResumeRecoveryJobs()	
Functional Description	
This service allows the FEE to execute internal recovery jobs.	
	Info Without suspending recovery jobs first, calling this service has no effect.
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.12 Fee_30_FlexNor_ResumeRecoveryJobs

4.1.13 Fee_30_FlexNor_SuspendFlashWrites

Prototype	
void Fee_30_FlexNor_SuspendFlashWrites ()	
Functional Description	
This service blocks the FEE to request write/erase jobs from the lower layer.	
	Info
	Ongoing write related jobs as well as read jobs of the lower layer are not affected. If the FEE has to request a write related job from the lower layer to finish its job, it will stay in state <code>MEMIF_BUSY</code> until write/erase jobs are allowed or the current job is aborted.
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.13 Fee_30_FlexNor_SuspendFlashWrites

4.1.14 Fee_30_FlexNor_ResumeFlashWrites

Prototype	
void Fee_30_FlexNor_ResumeFlashWrites ()	
Functional Description	
This service allows the FEE to request write/erase jobs from the lower layer.	
	Info
	Without suspending write related jobs first, calling this service has no effect.
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.14 Fee_30_FlexNor_ResumeFlashWrites

4.1.15 Fee_30_FlexNor_GetEraseCycleCounter

Prototype	
Std_ReturnType Fee_30_FlexNor_GetEraseCycleCounter (Fee_30_FlexNor_PartitionIdType PartitionId, Fee_30_FlexNor_EraseCycleCounterType* EraseCycleCounter)	
Parameters	
PartitionId [in]	Id of the partition whose maximal ECC shall be returned.
EraseCycleCounter [in]	Pointer to buffer from upper layer where the result is written into.
Return Codes	
E_OK The job was accepted E_NOT_OK The job was rejected	
Functional Description	
This service returns the maximal erase cycle counter (ECC) of all sectors belonging to the provided partition. Additionally, if development mode is configured, parameter checks are done and in case of failure they are reported to the DET by default with the according service ID and the reason of occurrence (refer to chapter Section 2.3.1).	
Limitations	
> This service is only available after the provided partition was already started up.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.15 Fee_30_FlexNor_GetEraseCycleCounter

4.1.16 Fee_30_FlexNor_MainFunction

Prototype	
void Fee_30_FlexNor_MainFunction ()	
Functional Description	
This service triggers the processing of the internal state machine and handles the asynchronous job and management operations. The complete handling of the job and the detection of invalidated or inconsistent blocks will be done in the internal job state machine.	
Limitations	
> Required to be called in same context as NVM MainFunction.	
Particularities	
> This service is always available.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.16 Fee_30_FlexNor_MainFunction

4.1.17 Fee_30_FlexNor_GetVersionInfo

Prototype	
void Fee_30_FlexNor_GetVersionInfo (Fee_30_FlexNor_VersionInfoPtrType versioninfo)	
Parameters	
versioninfo [in]	Pointer to where to store the version information.
Functional Description	
Returns the version information, vendor ID and AUTOSAR module ID of the component.	
Particularities	
> This service is configurable	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.17 Fee_30_FlexNor_GetVersionInfo

4.1.18 Fee_30_FlexNor_JobEndNotification

Prototype	
void Fee_30_FlexNor_JobEndNotification ()	
Functional Description	
This routine shall be called by the underlying flash driver to report the successful end of an asynchronous operation.	
	Caution
	While this parameter may be configurable within the referenced Flash driver, the callback name must be configured to the concrete name <code>Fee_30_FlexNor_JobEndNotification()</code> and may not differ.
Particularities	
> This service is always available.	
Options	
callcontext: This routine might be called on interrupt level, depending on the calling function. reentrant: FALSE synchronous: TRUE	

Table 4.18 Fee_30_FlexNor_JobEndNotification

4.1.19 Fee_30_FlexNor_JobErrorNotification

Prototype
void Fee_30_FlexNor_JobErrorNotification ()

continues on next page

Table 4.19 – continued from previous page

Functional Description	
This routine shall be called by the underlying flash driver to report the failure of an asynchronous operation.	
	Caution While this parameter may be configurable within the referenced Flash driver, the callback name must be configured to the concrete name <code>Fee_30_FlexNor_JobErrorNotification()</code> and may not differ.
Particularities	
> This service is always available.	
Options	
callcontext: This routine might be called on interrupt level, depending on the calling function. reentrant: FALSE synchronous: TRUE	

Table 4.19 Fee_30_FlexNor_JobErrorNotification

4.1.20 Fee_30_FlexNor_AddressAreaJobEndNotification

Prototype	
void Fee_30_FlexNor_AddressAreaJobEndNotification (MemAcc_AddressAreaIdType addressAreaId,, MemAcc_JobResultType jobResult)	
Parameters	
addressAreaId, [in]	Id of the Fees AddressArea.
jobResult [in]	Result of the finished job.
Functional Description	
This routine shall be called by the underlying MEMACC module if a job is finished. The job result is directly forwarded as a parameter.	
	Caution While this parameter may be configurable within the referenced MEMACC module, the callback name must be configured to the concrete name <code>Fee_30_FlexNor_AddressAreaJobEndNotification</code> and may not differ.
Particularities	
> This service is always available.	
Options	
callcontext: This routine might be called on interrupt level, depending on the calling function. reentrant: FALSE synchronous: TRUE	

Table 4.20 Fee_30_FlexNor_AddressAreaJobEndNotification

4.2 Required Functions

The module requires these functions from other modules. Refer to their documentation for API details.

Module	API
Det	Det_ReportError (optionally) Det_ReportRuntimeError (optionally)
FLS (if used instead of MEMACC)	Fls_Read Fls_Write Fls_Erase Fls_GetStatus (if polling mode activated) Fls_GetJobResult (if polling mode activated) Fls_Cancel Fls_BlankCheck (optionally)
MEMACC (if used instead of FLS)	MemAcc_Read MemAcc_Write MemAcc_Erase MemAcc_GetJobStatus (if polling mode activated) MemAcc_GetJobResult (if polling mode activated) MemAcc_Cancel MemAcc_BlankCheck (optionally)
NVM	NvM_JobEndNotification (optionally) NvM_JobErrorNotification (optionally)
User	Fee_30_FlexNor_DiagnosticHandler_UserCallback (optionally)

Table 4.21 Required Functions

5 Configuration

The functionalities of the module are configured with DaVinci Configurator Classic.

For details on configuration options refer to the module-specific description that is shown in DaVinci Configurator Classic.

5.1 Configuration Variants

This module supports these configuration variants:

> VARIANT-PRE-COMPILE

The configuration classes of the parameters depend on the supported configuration variants. For their definitions please see the module-specific BSWMD ARXML file.

5.2 Configuration with DaVinci Configurator

5.2.1 Block Configuration

The FEE uses statically configured blocks to manage data. A FEE block can be configured as stand-alone block but is typically linked to an NVM block.

5.2.1.1 BlockNumber and BlockId

The `FeeBlockNumber` is used to identify the block when a FEE API is called. The block number must be unique within the whole FEE block configuration.

- > The assigned block numbers are spaced by the possible number of datasets that is derived from the `NvMDatasetSelectionBits` attribute of the `NvM Common` configuration. In case no NVM configuration is available, four dataset selection bits are assumed.

The `FeeBlockId` is used to identify the block in flash. The block id must be unique within the FEE partition configuration.

- > When the block ids are changed due to a configuration update or an optimization, the data that was already stored in the flash for the changed blocks is lost.

5.2.1.2 Dataset Blocks

The NVM provides the concepts of redundant and dataset blocks where additional versions of the same NVM block are stored in flash. These blocks are mapped to the FEE configuration by a dedicated FEE block for each required dataset. The relation between these blocks is established using the `FeeBlockNumber` attribute.

**Example**

Assuming the NVM configuration contains the block `NvMBlock_Dataset_Example` that is configured as dataset block with four datasets
The correct FEE configuration then contains the following blocks:

Name	BlockNumber	NVM Reference
FeeBlock_Dataset_Example	16	The NVM references this block.
FeeBlock_Dataset_Example_01	17	
FeeBlock_Dataset_Example_02	18	
FeeBlock_Dataset_Example_03	19	

Table 5.1 FEE dataset example configuration

The names of the blocks are derived from the NVM block name, but the additional dataset blocks have a suffix with an increasing number to make the name unique. The block number of the first block is aligned to a multiple of the spacing defined by the `NvMDatasetSelectionBits` configuration. The block numbers of the residual dataset block are then assigned successively by always incrementing the block number by one.

**Note**

The FEE block reference of the configured NVM block must always point to the first block whose block number is aligned to the spacing that was derived from the `NvMDatasetSelectionBits`.

5.2.1.3 Lookup Table Blocks

To be able to persist the current state of the lookup tables that are held in RAM, the block configuration must contain a lookup table block for each configured partition.

A lookup table block has the same configuration parameters as a regular user block, but its block id is always fixed at 1, hence, it's the first block in each partition. The size of the lookup table block is automatically determined and set, depending on the number of other blocks in the partition using the following formular:

$$\text{lookup table block size} = (\text{number of blocks in partition}) \cdot 4 \text{ bytes}$$

**Note**

The existence of the lookup table blocks is ensured by validation in case the FEE is configured using the DaVinci configurator. Missing lookup table blocks can then be created automatically using the suggested solving action.

5.2.2 Partition Configuration

The FEE allows the configuration of multiple partitions that are used to manage blocks independently from each other. For more information about partitions, see Section 2.5.1.

5.2.2.1 Reference to Flash Access Layer

Within `FeePartitionTarget` either a reference to a FLS or to a MEMACC address area can be set. As FEE does not allow parallel usage of FLS and MEMACC module all configured partitions must either reference FLS or the same MEMACC address area. Note that the address area configured within MEMACC module and referenced by FEE must be configured properly:

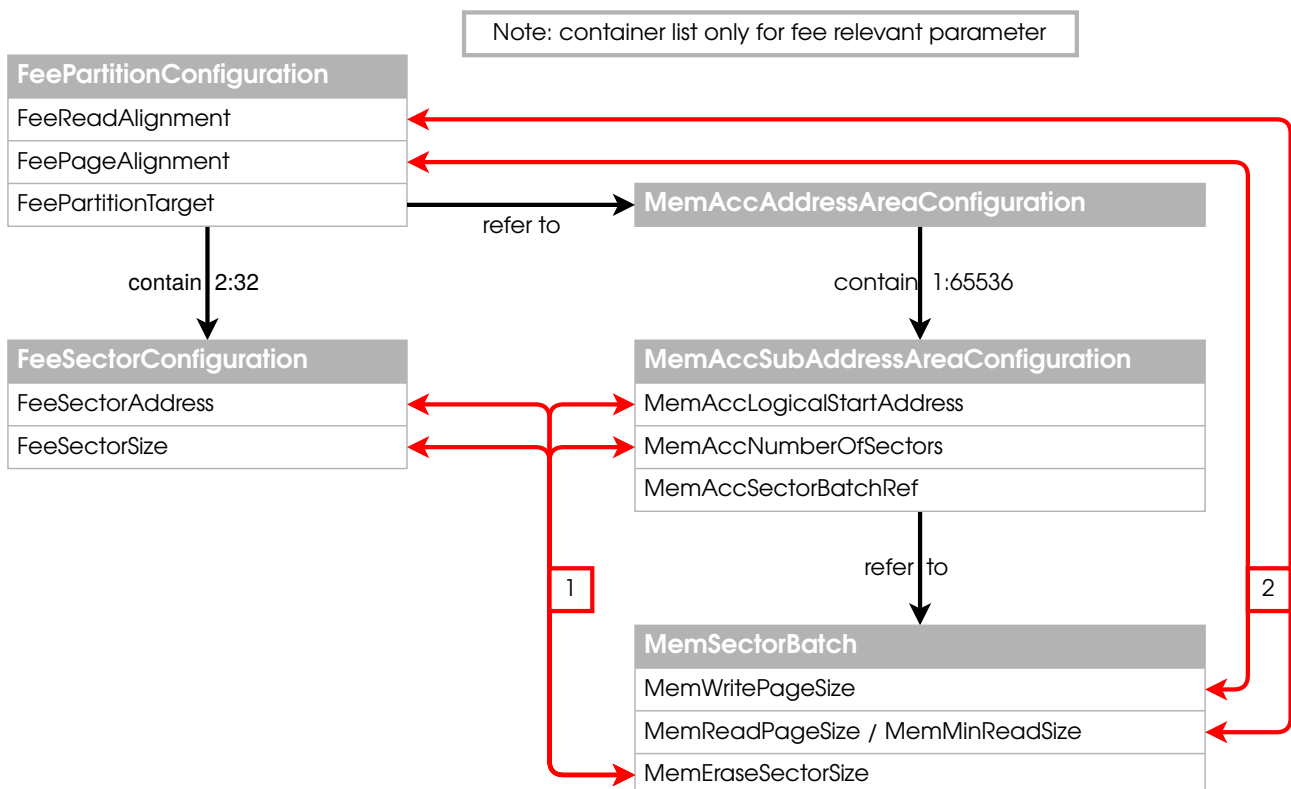


Figure 5.1 Configuration Overview: Reference to MEMACC

1. The `FeeSectorConfiguration` must be covered by the logical memory space provided by the referenced MEMACC Address Area which is summed up by the logical memory space provided by each configured Sub Address Area. The logical memory space of each configured Sub AddressArea spans from `MemAccLogicalStartAddress` with the size calculated by $\text{MemAccNumberOfSectors} * \text{MemEraseSectorSize}$.
2. The `FeePageAlignment` must be configured to be at least the lowest common multiple of all referenced `MemWritePageSize` as FEE will build up the memory image for a whole partition based on `FeePageAlignment` value. The same is the case for

`FeeReadAlignment` and `MemReadPageSize` or `MemMinReadSize` depending on the supported AUTOSAR version of the used Mem.

Correct configuration will be ensured and supported by validators.

**Note**

The configured `Mem/MemPublishedInformation/MemErasedValue` must be the same for all Mem driver used by FEE.

5.2.2.2 Alignment Information

`FeeReadAlignment` specifies the minimal amount of data that can be read on one read request.

- > The Fee will align all read requests according to this value. The process of aligning the reads can cause the payload read operation to be split into up to 4 read requests to the MemAcc to fulfill this alignment. The exact number of read requests depends on the payload size, read length, offset, and the configured `FeeReadAlignment`. Setting an alignment of one reduces this to a maximum of 1-2 read requests. A small read alignment therefore can decrease the number of read requests to the MemAcc component and increase the amount of bytes requested at once.
- > To maximize the read performance, a small `FeeReadAlignment` to minimize the Fee requests to the MemAcc and a high `MemReadBurstSize` or `MemMaxReadSize` to minimize the MemAcc requests to Mem are recommended.

`FeePageAlignment` specifies the minimal amount of data that can be written in one write request.

- > The page alignment is used internally to align the inner parts of the structures management data. A large page size therefore leads to an increased memory consumption by the FEE management data.

`FeeInterferenceFreeAlignment` specifies an alignment that ensures that data that was stored in two different alignment areas are not influenced by each other.

- > Some flash devices are designed in a way, that read or write operations in one page can have influence on adjacent pages. Therefore, the interference free alignment is required to ensure that parts of the metadata doesn't interfere with each other.
- > The interference free alignment must be configured carefully. If the alignment is too small, data loss may occur because erroneous pages can also invalidate adjacent data. If the alignment is too large, memory is wasted because all metadata structures are aligned according to the interference free alignment.

An example for a platform where the `FeeInterferenceFreeAlignment` is not equal to the `FeePageAlignment` would be NXP MPC57xx. E.g., with a

- > Page alignment of 8 byte
- > Interference free alignment of 32 byte

The information can be normally found in the memory related chapters of the platforms data sheet.

5.2.2.3 Logical Sectors

Each partition consists of 2 to 32 logical flash sectors that are laid over multiple physical flash sectors. For details about the functional concept see Section 2.5.2.

5.2.3 FEE General Configuration

5.2.3.1 Internal Buffer Size

For some operations, FEE cannot use the given user buffer, but has to use an own internal buffer, e.g. for reading/writing management data and especially for copying user data during a garbage collection. For copying, FEE alternately reads and writes from the internal buffer, until the complete payload has been copied. Depending on the size of the buffer and the payload size, several steps might be necessary. The user can configure the size of this internal buffer. Following implications can be made:

- > The size of the buffer must be page aligned.
- > As a minimum, the buffer size must be bigger or equal to the size of the biggest management entity (i.e. multiple of `FeeInterferenceFreeAlignment`). The correct minimum size will be ensured by validation.
- > The smaller the buffer, the more steps are necessary for copying a block (also dependent on the payload size), the longer the garbage collection will take, but the less RAM is needed.
- > The larger the buffer, the less steps are necessary for copying a block (also dependent on the payload size), the shorter the garbage collection will take, but the more RAM is needed.
- > Configuring the buffer size bigger than the biggest payload has no effect on the performance and just increases the RAM usage. Hence, it is not recommended to do so.



Caution

Due to above restrictions choose the internal buffer size with care. Users can also test their system with different buffer sizes since this has no influence on the flash layout compatibility.

5.2.4 Flash API Configuration

The FEE configuration provides two options for the configuration of the flash API.

1. Use the default AUTOSAR flash function names. These are encoded in the FEE generator and no explicit configuration is necessary.
2. Use the optional `FeeFlsApi` sub container of the `FeeGeneral` container to configure the flash function names, as well as the name of the flash include file explicitly.

5.2.4.1 Fls_BlankCheck

If a partition is configured to execute a blank check before reading from flash, the blank check API must be available.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator	Generation tool for MICROSAR Classic modules
Block Tag	Identifies a block in data flash. See also: Block Id
Block Id	Identifier used to uniquely identify blocks in a partition. Stored in data flash, it consists of Block Tag and Data Index. In conjunction with partition ID, every block in FEE can be uniquely identified. The Block Id abstracts from 16Bit Block Number passed via API; it depends on current configuration, in particular on Dataset Selection Bits. The block tag is one cornerstone in FEE's update capability: As long as a Block Id does not change across configuration updates, it can be found in data flash, data may be kept, if block length also remained unchanged.
MEMACC Address Area	An AddressArea is a logical area of memory. FEE use logical addresses to access the address area. An AddressArea contains SubAddressAreas and each SubAddress Area is part of a physically continuous memory area. For more information see [6].

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read Only Memory
FEE	Flash EEPROM Emulation Module
FLS	Flash Driver
ISR	Interrupt Service Routine
LUT	Look Up Table
MEMACC	Memory Access
MEM	Memory driver
MEMIF	Memory Abstraction Interface Module
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)

continues on next page

Table 6.2 – continued from previous page

Abbreviation	Description
NVM	NVRAM Manager
NVRAM	Non Volatile Random Access Memory
SchM	(AUTOSAR) Scheduling Manager
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification
VMEMACCM	Vector Memory Access Manager

Table 6.2 [Abbreviations]

7 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com